

A Neuromorphic FPGA-based Architecture for Energy-Efficient Visual Recognition

Tianyang Li¹, Zhipeng Liao^{1,2,3}, Chengjun Zhang¹, Jie Yang^{1,2,3}, Mohamad Sawan^{1,2,3}

¹Westlake Institute for Optoelectronics, Fuyang, Hangzhou, China

²CenBRAIN Neurotech Center of Excellence, Westlake University, Hangzhou, China

³Integrated-On-Chips Brain-Computer Interfaces Zhejiang Engineering Research Center, Hangzhou, China

Abstract—Neuromorphic computing has emerged as a promising paradigm for enabling energy-efficient and real-time machine intelligence on edge platforms. In this work, we present a neuromorphic computing approach for visual recognition by implementing a Spiking-YOLO algorithm on a custom FPGA platform. The conventional YOLO network is converted into a spiking neural network, where event-driven architectures enable efficient feature extraction and bounding box prediction. On FPGA, we design optimized modules for membrane potential integration, spike generation, and parallel convolution, combined with weight quantization and memory-efficient dataflow. This implementation achieves over 50× energy savings compared to digital CNN accelerators, while achieving over 20 FPS real-time object detection performance with minimal accuracy loss. Integrated with vision sensors, the system demonstrates an energy-efficient, compact, and adaptive solution for edge AI applications such as robotics, surveillance cameras, and autonomous systems. Our results highlight the potential of neuromorphic chips in advancing scalable and low-power visual perception.

Index Terms—Neuromorphic computing, Spiking-YOLO, FPGA, Quantization, Energy-Efficient.

I. INTRODUCTION

Running deep learning-based visual recognition algorithms on edge devices has been widely applied in fields such as autonomous driving and industrial automation [1]. However, the high energy consumption during edge-side operation further limits its development. As the second generation of neural networks, Spiking Neural Networks (SNNs) utilize spikes instead of features for information transmission and employ accumulation operations instead of traditional multiply-accumulate operations. Meanwhile, SNNs can achieve up to 90% input sparsity without requiring pruning [2]. These characteristics enable SNNs to operate with extremely low power consumption, making them an ideal choice for deployment on edge devices.

Recent applications of SNNs have expanded from small-scale tasks such as digit recognition or simple serial processing [3], [4] to more challenging domains including object detection [5], image recognition [6], and semantic segmentation [7]. Through modifications to modern neural network architectures like YOLO and Transformer, combined with appropriate training and conversion methods, the performance of SNNs in complex tasks has been significantly improved. For instance, the introduction of temporal encoding mechanisms and dynamic threshold adjustment strategies enables SNNs to process temporal information and high-dimensional inputs more efficiently [8].

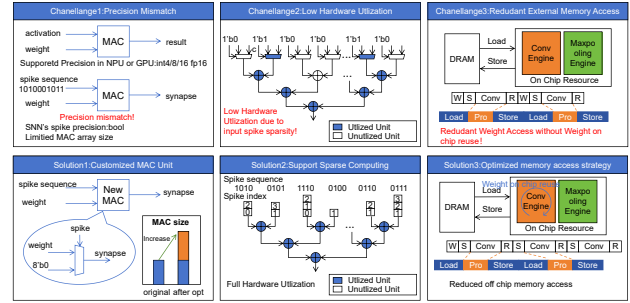


Fig. 1. Challenges and Solutions for SNN Deployment on Neuromorphic Processors

Coupled with hardware design optimization, the deployment efficiency and applicability of SNNs on edge devices have been further enhanced, demonstrating substantial potential for practical applications.

As shown in Fig. 1, the deployment of SNNs on GPUs and NPUs faces the following challenges:

- (1) The inherent MAC (Multiply-Accumulate) units in GPUs and NPUs are incompatible with the accumulation operations in SNNs, resulting in low hardware energy efficiency [9]. This incompatibility also limits the scale of neuron deployment in SNNs, leading to high latency during edge-side operation and failure to meet real-time requirements for certain tasks.
- (2) Due to the parallel processing architecture of GPUs and NPUs, coupled with the highly sparse nature of SNNs, these hardware platforms cannot effectively leverage the sparsity of SNNs for computation [9]. This introduces unnecessary computations, low hardware resource utilization, and additional latency and power consumption.
- (3) In the current input data formats of GPUs and NPUs, a single spike occupies multiple bits, resulting in inefficient use of storage resources. The compilation results from GPUs and NPUs cannot effectively reuse input weights. Additionally, due to the data dependency of neuron states across timesteps, partial sum results need to be frequently transferred between off-chip and on-chip memories, leading to increased external memory access overhead.

FPGAs possess flexible reconfigurability that can fully accommodate the characteristics of SNNs, serving as a potential solution for accelerating spiking neural networks. This has been validated in previous SNN model studies [11]–[13]. In this

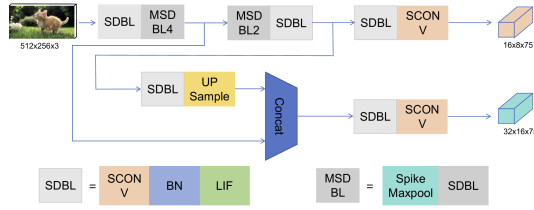


Fig. 2. Spiking YOLOv3-tiny network architecture

paper, we implement a neuromorphic computing framework on FPGA and efficiently run the Spiking-YOLO model [14] on this framework. Compared to traditional neural network accelerators, the energy consumption is only 1/50, while achieving a processing speed of 20 FPS, meeting real-time requirements.

As shown in Fig. 1, the neuromorphic computing framework features the following characteristics:

- (1) Employing multiplexers (Mux) instead of multiplication operations in MAC units, reducing on-chip resource utilization, increasing the number of supported hardware neurons, improving neuron deployment scale, and consequently enhancing processing speed.
- (2) Designing a vector processing unit (VPU) that simultaneously supports both sparse and dense operations. The VPU utilizes spike positions to index weights, fully leveraging the sparsity of input spikes while improving hardware resource utilization efficiency.
- (3) Compressing input spikes so that each spike occupies only one bit in memory. Additionally, the framework employs a RISC-V ISA-based CPU as a scheduler, enabling flexible management of various resources within the framework. Leveraging the scheduler's reconfigurability, on-chip weights are highly reused, reducing external memory overhead.

Fig. 1 illustrates the challenges currently faced by SNN deployment on existing neuromorphic processors and presents corresponding solutions to the aforementioned problems.

II. HARDWARE ARCHITECTURE

A. Overall Architecture

We designed a high-performance neuromorphic processing framework based on FPGA, which accelerates Spiking-YOLO by fully utilizing FPGA resources. As shown in Fig. 3, the overall hardware architecture of the neuromorphic processing framework mainly consists of a CPU, DMA, memory controller, multiple computing cores, max-pooling processing units, high-performance inter-core accumulation units, and data arrangement units.

This accelerator employs multi-core parallel processing to accomplish convolutional tasks in Spiking-YOLO. The CPU is responsible for distributing tasks to different computing cores and synchronizing their operations. The DMA handles data transfer between external memory and on-chip memory. Each computing core comprises the following components: Sparse/Dense Vector Processing Unit (S/D VPU), on-chip

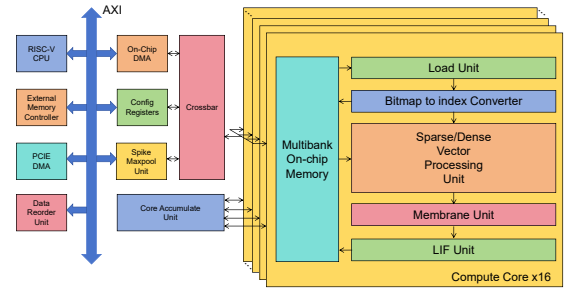


Fig. 3. Hardware architecture of the proposed neuromorphic processing framework

memory, Loading Unit (LDU), Bitmap-to-Index Conversion Unit, Membrane Potential Storage Unit, and LIF Unit.

The Sparse/Dense Vector Processing Unit serves as the core component of the computing core, primarily responsible for mapping convolution operations to vector-matrix multiplication (including both sparse and dense vectors). The on-chip memory stores input weights, spikes, and output results. During core operation, the Loading Unit transfers spikes from on-chip memory to the computing array. The Bitmap-to-Index Conversion Unit is mainly used for zero-skipping operations in sparse computations. The Membrane Potential Storage Unit and LIF Unit are responsible for storing neuronal membrane potentials across different timesteps and generating spikes at respective timesteps. The generated spikes are subsequently stored in the on-chip memory.

The max-pooling processing unit handles the maximum suppression operations in Spiking-YOLO. The data arrangement unit reorganizes data in off-chip memory, enabling the processing results from one layer in Spiking-YOLO to be fed into the next layer for subsequent computation. The inter-core accumulation unit, composed of an on-chip ring bus, facilitates the summation of membrane potentials across multiple computing cores, thereby eliminating the limitation of a maximum of 128 input channels per single computing core.

B. Sparse/Dense Vector Processing Unit

Although the high sparsity of input spikes in SNNs can significantly reduce energy consumption, existing GPU and NPU architectures struggle to directly leverage this high sparsity [15], leading to low utilization rates of on-chip MAC units. To avoid unnecessary computations while improving the utilization efficiency of on-chip computing units and fully exploiting the benefits of sparsity techniques, we designed an S/D VPU to comprehensively handle all convolution-related operations, including sparse and dense convolutions for all kernel sizes. As shown in Fig. 4, the S/D VPU contains multiple VAUs. These units fetch weights from on-chip memory through streaming transmission and perform accumulation operations. The accumulated results are output as intermediate values for each output channel to the membrane potential storage unit, where they are summed with the corresponding membrane potentials. By configuring the S/D VPU, it can operate in either sparse computation mode or dense computation mode. In sparse computation mode, the external bitmap-to-index conversion unit

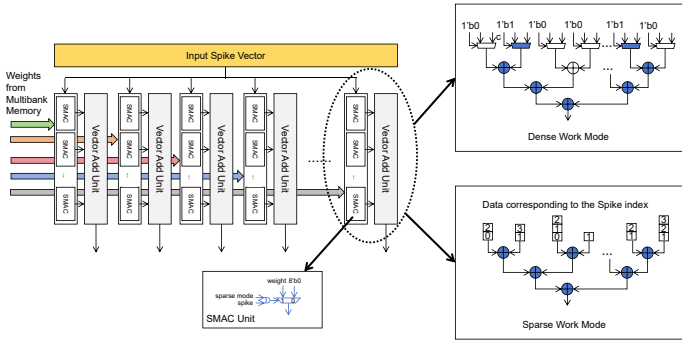


Fig. 4. Architecture of the S/D VPU

divides the 128 input channels into 16 groups, with each group containing at most 8 spikes. Based on the spikes, it generates 16 sets of indices and sends them to the loading unit. The loading unit then reads the weight data corresponding to the spike indices from multiple banks of on-chip memory in parallel. Each row of weight data contains 16 output channels. After transposing the input data from on-chip memory, the S/D VPU feeds it into the VPU, ultimately generating 16 output channels. In dense computation mode, the read addresses of each bank in on-chip memory are pre-locked, and the weights output from on-chip memory remain constant during computation. The loading unit loads spikes from 16 input channels, which are then fed into the selection bits of multiplexers to determine whether the weights from on-chip memory should be sent to the Vector Addition Units for computation. This process finally generates 16 output channels.

C. Bitmap to index Converter

Fig. 5 illustrates the architecture of the bitmap-to-index converter. Each converter consists of a fixed-priority arbiter, a one-hot-to-binary conversion unit, and a handshake unit. The converter receives an 8-bit bitmap input, which serves as the request input to the fixed-priority arbiter. The fixed-priority arbiter outputs a one-hot code, which is then converted into a spike index through the one-hot-to-binary conversion unit. After each conversion is completed, the handshake unit performs an XOR operation between the generated one-hot code and the input bitmap to deassert the previous request, thereby enabling the next bitmap-to-index conversion.

A single computing core incorporates 16 such converters, capable of processing spikes from 128 input channels simultaneously. This module generates addresses for accessing 16 weight memory banks each time, reading 16 weight values per cycle corresponding to 16 non-zero input bits. These values are then fed into subsequent adder trees for accumulation operations. By employing this approach, the sparsity of SNN input spikes is effectively utilized through zero-skipping operations to avoid unnecessary computations, thereby reducing latency associated with processing zero-bit inputs. Furthermore, the 16-way parallelism ensures high efficiency in spike conversion.

D. Core Accumulate Unit

Since a single computing core can handle a maximum of 128 input channels, while the Spiking-YOLO model requires

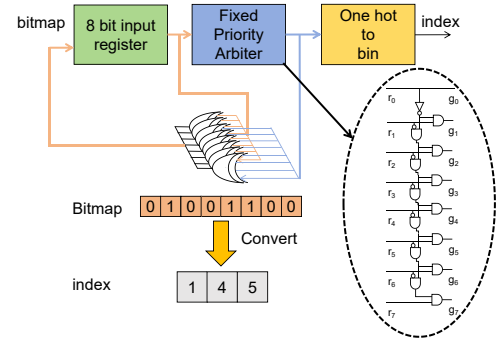


Fig. 5. Architecture of the bitmap-to-index converter

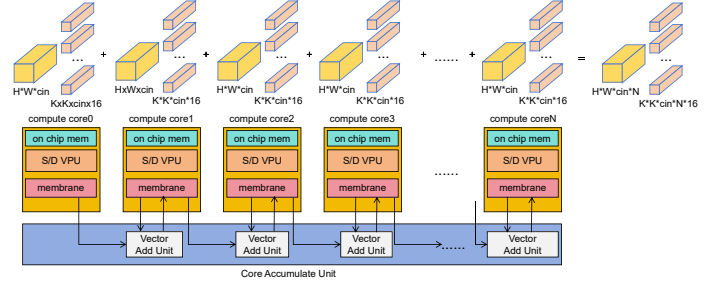


Fig. 6. Architecture of the inter-core accumulation unit with ring bus topology

up to 1024 input channels, it is essential to introduce inter-core communication for membrane potential accumulation to reduce external memory access. Fig. 6 shows the architecture of the inter-core accumulation unit.

This module connects all computing cores in a ring bus topology. The ring bus supports membrane potential accumulation between adjacent computing cores. The connection pattern between computing cores can be configured through external configuration registers, allowing grouping in units of 2^N cores. The membrane potential result for each group is obtained by accumulating the membrane potentials from 2^N cores.

This module operates after convolution computations are completed. The read units connecting adjacent computing cores are responsible for reading the local membrane potential and the membrane potential from neighboring cores. After accumulation, the result is written back to the local computing core. This approach enables on-chip expansion of input channel capacity. The maximum number of input channels supported by this framework depends on the number of computing cores.

III. OFF CHIP MEM ACCESS OPTIMIZATION

A. Weight On Chip Reuse

During the convolution phase, the primary efficiency bottleneck arises from the frequent access to convolution weights from off-chip memory to expand the number of input channels. To reduce the demand for weight vector access from off-chip memory, we strive to avoid repeatedly loading the same weights during the scheduling phase.

Since this framework supports multi-channel membrane potential fusion, it can overcome the limitation of 128 input convolution channels per single computing core. Additionally,

Algorithm 1 Weight on Chip Reuse Scheduling Method

```

1: Input: spike_in, Weight
2: Output: spike_out
3: // Serial compute
4: for  $C_{out} = 0$  to  $(C_{out\_max}/16)$  do
5:   Load_weight_from_off_chip_memory;
6:   for  $H = 0$  to  $((H_{max} - K + 1)/Div\_H)$  do
7:     for  $W = 0$  to  $((W_{max} - K + 1)/Div\_W)$  do
8:       for  $t = 0$  to  $T$  do
9:         for  $C_{in} = 0$  to  $((C_{in\_max})/(128 \times N\_PE))$  do
10:          Load_activation_from_off_chip_memory;
11:          // Parallel compute between PE cores
12:          for  $PE = 0$  to  $N\_PE$  do ▷ Parallel loop
13:            if  $T == 0$  then
14:              Clear_memb(Memb[PE]);
15:            end if
16:            // Serial compute in single PE core
17:            for  $K_h = 0$  to  $K$  do
18:              for  $K_w = 0$  to  $K$  do
19:                for  $H_{div} = 0$  to  $Div\_H - K + 1$  do
20:                  for  $W_{div} = 0$  to  $Div\_W - K + 1$  do
21:                    // Parallel compute
22:                    for  $C_{out} = 0$  to 15 do
23:                      // Parallelism 16 (dense)
24:                      for  $C_{in} = 0$  to 127 do
25:                        // Parallelism 128 (sparse)
26:                         $Memb[PE][H_{div}][W_{div}][C_{out}] +=$ 
27:                           $Weight[PE][K_h][K_w][C_{in}][C_{out}]$ 
28:                           $\times Spike\_in[PE][H_{div}][W_{div}][C_{in}];$ 
29:                      end for
30:                    end for
31:                  end for
32:                end for
33:              end for
34:            end for
35:          end for
36:          // Execute memb_accumulate between PE cores
37:          // Serial Compute
38:          for  $PE = 0$  to  $N\_PE$  do
39:            Accumulate(Memb[PE], Memb[PE + 1]);
40:          end for
41:          if  $C_{in} == (C_{in\_max})/(128 \times N\_PE) - 1$  then
42:            // Leaky integrate-and-fire and store to result buffer
43:            LIF(Memb[N\_PE]);
44:            Store_to_off_chip_mem(spike_out[N\_PE]);
45:          end if
46:        end for
47:      end for
48:    end for
49:  end for
50: end for

```

during the convolution phase, the weight values for each computing core are small data vectors that can be fully accommodated by the FPGA's on-chip buffers. Therefore, to minimize frequent read/write operations of convolution weights in off-chip memory, we distribute convolution layers with more than 128 input channels across multiple computing cores. Finally, after each spiking convolution operation, the membrane potentials within the computing cores are accumulated, and spike generation operations are performed in the accumulating cores. The generated spikes are then written back to off-chip memory.

Algorithm 1 presents a scheduling strategy for maximum weight reuse. This strategy features a maximum parallelism of 16 output channels, requiring only that the weights for 16 outputs be evenly distributed across all computing cores in the neuromorphic processing framework. Since the weight loading phase is at the outermost level of the scheduling loop, the number of weight loading operations depends only on the number of output channels, thereby avoiding repeated

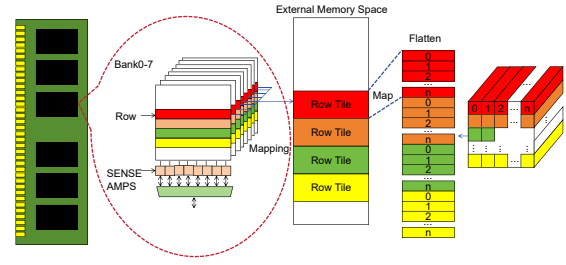


Fig. 7. Optimized data layout in off-chip memory to avoid cross-row access

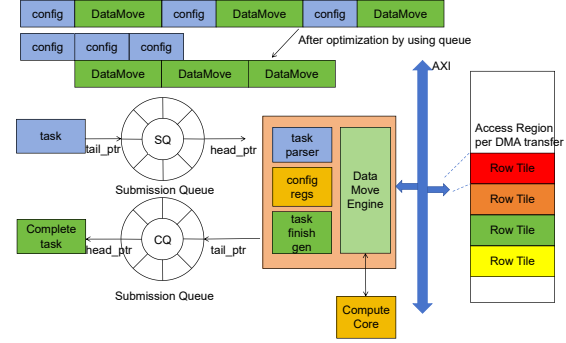


Fig. 8. Architecture of the task queue-based DMA with dual queues

weight loading from off-chip memory. This strategy reduces unnecessary memory access while lowering system bandwidth requirements and data transmission overhead.

B. Reduce Memory Access Time

Off-chip memory requires periodic refresh operations on row storage units to maintain data integrity due to its inherent hardware characteristics. Consequently, accessing off-chip memory introduces additional latency and energy [16], making optimization of off-chip memory data access essential. This paper leverages the hardware characteristics of off-chip memory and optimizes the on-chip hardware architecture to improve off-chip memory access bandwidth.

Cross-row access in off-chip memory requires additional refresh time, which significantly impacts data access bandwidth. When on-chip DMA operations involve cross-row access, row switching operations introduce extra latency, resulting in reduced external memory bandwidth. Therefore, during single off-chip data access operations, cross-row data access should be minimized.

In the entire model, the maximum weight input size for a single layer is 4 MB, and the maximum input spike size for a single layer is 256 KB. However, the maximum data capacity per row in off-chip memory is 64 KB. If weights and spikes are arranged sequentially in contiguous blocks, cross-row access issues will occur. Thus, input data must be partitioned into multiple blocks based on the maximum storage capacity per row and rearranged accordingly. This ensures that on-chip DMA operations avoid cross-row access, thereby improving data bandwidth. The data arrangement strategy is illustrated in Fig. 7.

To avoid the additional latency caused by configuring DMA registers during memory access, we employ a task queue-based

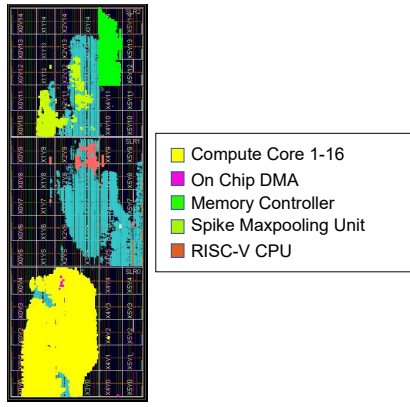


Fig. 9. Place-and-route results of the neuromorphic computing framework

DMA in this framework. As shown in Fig. 8, the task queue-based DMA consists of a request queue, a completion queue, and a memory transfer unit.

During DMA operation, the CPU writes configuration parameters from the memory transfer unit as tasks into the request queue and periodically checks whether the completion queue is non-empty. When the DMA is idle or has completed a task, it reads task units from the request queue and executes DMA operations. Upon completion of a DMA operation, the operation results are written back to the completion queue. Each DMA transfer should avoid accessing cross-row units to minimize additional memory access time.

IV. EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS

We deployed the neuromorphic computing framework on a Xilinx XCVU9P-FLGA2104 FPGA. The entire neuromorphic computing framework operates at 100MHz, with FPGA resource utilization shown in Table II and the synthesis and place-and-route results illustrated in Fig. 9.

The design utilizes 273,638 lookup tables (LUTs), 287,316 flip-flops (FFs), 348 BRAMs, 48 URAMs, and 711 DSPs. In the place-and-route results, the yellow regions represent the 16 computing cores, which occupy the largest area. The green regions correspond to the external memory controller, which occupies the second largest area. The CPU and DMA occupy relatively smaller areas. The dynamic power consumption of the SNN computing framework portion in the overall design is only 4.1W.

The system processes input images with a resolution of 256×128 pixels through a deep feature extraction network for object detection tasks. In the first five layers of this network, the number of output channels doubles, and the feature map size gradually decreases after downsampling operations by max-pooling layers. In the deeper Conv6 layer, the number of channels increases to 1024 to obtain rich semantic feature representations. In subsequent network layers, the number of output channels is reduced, while the feature map size remains unchanged or undergoes upsampling, ultimately producing two output feature maps with 75 channels each. The Conv11 layer

employs a feature fusion mechanism, effectively integrating multi-scale features through concatenation operations.

This experiment statistics the runtime of each layer in both dense and sparse modes, obtaining the total runtime of the entire network, as shown in Table III. Since the input channels of the first three layers do not meet the requirements for dense mode operation, their runtimes are identical in both modes.

To evaluate the effectiveness of the proposed architecture, we compare the experimental results with several state-of-the-art schemes. As shown in Table I, our FPGA design achieves the highest processing speed of 20 FPS for 256×128 resolution images at a clock frequency of 100MHz operating in sparse mode, while consuming only 3.2W, achieving an excellent balance between performance and power consumption.

Compared to other FPGA implementation schemes, the work in [17] achieves 69 FPS (15.4W power consumption) and 55 FPS (13.6W power consumption) [18] at different resolutions, while our design finds a better balance between processing speed and hardware power consumption. Although GPU-based solutions can achieve a higher processing speed of 232 FPS at 224×224 resolution [19], their energy efficiency is relatively low.

Although FPGAs cannot achieve GPU-level high-frequency clock operation and high frame rate inference capabilities, our FPGA design demonstrates superior advantages in energy efficiency and reconfigurability. The proposed architecture provides an energy-efficient solution for real-time object detection applications while maintaining competitive performance.

V. CONCLUSION

This study implements a Spiking-YOLO algorithm on a custom FPGA for neuromorphic visual recognition. Leveraging an event-driven architecture with co-designed hardware for quantization and memory-optimized dataflow, our solution achieves over 50% energy savings compared to digital CNNs while maintaining real-time performance. The system processes 256×128 images at 20 FPS on a 100MHz clock, consuming only 3.2W with minimal accuracy loss.

While this work successfully demonstrates a high-energy-efficiency solution, we acknowledge the challenges associated with its architectural complexity. Future work will focus on architectural simplification and exploring more modular designs to enhance scalability and reduce design overhead. To address potential bottlenecks from the crossbar interconnect and high static power from on-chip SRAM, we will investigate more scalable interconnects, such as hierarchical Networks-on-Chip (NoCs), and explore emerging memory technologies. Furthermore, we plan to develop adaptive mechanisms to efficiently handle both sparse and dense workloads, broadening the architecture's applicability. To lower the barrier for practical use, we will also focus on creating a more sophisticated and automated compiler toolchain to simplify network mapping. Finally, we plan to deploy this computational framework on an ASIC platform to further enhance energy efficiency performance.

TABLE I
PERFORMANCE COMPARISON WITH STATE-OF-THE-ART SNN ACCELERATORS

	This Work	[17]	[18]	[19]	[21]	[22]
Platform	Xilinx XCVU9P	Xilinx VCU110	Xilinx ZC706	GPU	Xilinx ZYNQ	Xilinx Ultra96 V2
Power (W)	3.2	15.4	13.6	170.0	2.75	5.44
FPS	20.0	69.0	55.0	232	1.2	19.2
Energy Efficiency (FPS/W)	6.25	4.50	4.00	1.36	0.43	3.52
Image Size	256x128	416x416	416x416	224x224	416x416	448x448
Frequency	100	220	200	1000	100	214
NN Model	Spiking-yolo	YOLO V3 Tiny	YOLO V4 Tiny	Sim YOLO V2	YOLO V4 Tiny	YOLO V3 Tiny
Precision	A1W8	A16W8	A8W5	A32W32	A16W16	A8W8

TABLE II
FPGA RESOURCE UTILIZATION

Resource Type	Utilized	Available	Utilization (%)
LUT	273,638	1,182,240	23.15
LUTRAM	12,095	591,120	2.04
FF	287,316	2,364,480	12.15
BRAM	348	2,160	16.11
URAM	48	960	5.00
DSP	711	6,840	10.39
BUFG	36	1,800	2.00

TABLE III
RUNTIME COMPARISON OF DENSE AND SPARSE MODES FOR EACH LAYER

Layer	Dense Mode (ms)	Sparse Mode (ms)
Conv1+maxpool	5.275	5.275
Conv2+maxpool	5.288	5.288
Conv3+maxpool	5.140	5.140
Conv4+maxpool	5.443	4.572
Conv5+maxpool	7.158	6.098
Conv6	10.482	9.014
Conv7	1.274	1.083
Conv8	2.863	2.519
Conv9	0.263	0.218
Conv10	0.348	0.296
Conv11	11.890	10.220
Conv12	0.877	0.745

ACKNOWLEDGMENT

This work was supported in part by the STI2030-Major Projects under Grant 2022ZD0208805, in part by the “Pioneer” and “Leading Goose” Research and Development Program of Zhejiang under Grant 2024C03002, and in part by the Key Project of Westlake Institute for Optoelectronics under Grant 2023GD004. (Corresponding authors: Jie Yang; Mohamad Sawan.)

REFERENCES

- [1] C. Gheorghe et al., “Analyzing Real-Time Object Detection with YOLO Algorithm in Automotive Applications: A Review,” *CMES-Computer Modeling in Engineering & Sciences*, vol. 141, no. 3, 2024.
- [2] Fang, Chaoming et al., “An Energy-Efficient Unstructured Sparsity-Aware Deep SNN Accelerator With 3-D Computation Array,” *IEEE Journal of Solid-State Circuits*, vol. 60, no. 3, pp. 977 - 989, 2024.
- [3] S. R. Kulkarni and B. Rajendran, “Spiking neural networks for handwritten digit recognition—Supervised learning and network optimization,” *Neural Networks*, vol. 103, pp. 118–127, July 2018.
- [4] Y. Luo, X. Wang, L. Zhang, and Y. Li, “EEG-based emotion classification using spiking neural networks,” *IEEE Access*, vol. 8, pp. 46007–46016, March 2020.
- [5] J. Qu, et al., “Spiking Neural Network for Ultralow-Latency and High-Accurate Object Detection,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 3, 2024, pp. 4934–4946.
- [6] R. Kabilan, and Muthukumaran. N, “A Neuromorphic Model for Image Recognition Using SNN,” *In: 2021 6th International Conference on Inventive Computation Technologies (ICICT)*, IEEE, 2021.
- [7] Y. Kim, J. Chough, and P. Panda, “Beyond classification: Directly training spiking neural networks for semantic segmentation,” *Neuromorphic Computing and Engineering*, vol. 2, no. 4, p. 044015, December 2022.
- [8] W. Wei et al., “Temporal-coded spiking neural networks with dynamic firing threshold: Learning with event-driven backpropagation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.
- [9] Fang, Chaoming and Shen, Ziyang and Tian, Fengshi and Yang, Jie and Sawan, Mohamad, “An Area-Efficient and Bit-Width Configurable Carry-Save Adder Tree for Spiking Transformers,” in *the IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025.
- [10] Fang, Chaoming et al., “A 28nm Spiking Vision Transformer Accelerator with Dual-Path Sparse Compute Core and EMA-free Self-Attention Engine for Embodied Intelligence,” *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, p. 1 - 13, 2025.
- [11] J. Han, X. Wang, Y. Zhang, and L. Liu, “Hardware implementation of spiking neural networks on FPGA,” *Tsinghua Science and Technology*, vol. 25, no. 4, pp. 479–486, August 2020.
- [12] S. Panchapakesan, Z. Fang, and J. Li, “SyncNN: Evaluating and accelerating spiking neural networks on FPGAs,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 15, no. 4, pp. 1–27, December 2022.
- [13] X. Ju, Y. Wang, Z. Chen, and L. Zhang, “An FPGA implementation of deep spiking neural networks for low-power and fast classification,” *Neural Computation*, vol. 32, no. 1, pp. 182–204, January 2020.
- [14] S. Kim et al., “Spiking-YOLO: Spiking Neural Network for Energy-Efficient Object Detection,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 7, 2020.
- [15] Z. Wang, “SparseRT: Accelerating Unstructured Sparsity on GPUs for Deep Learning Inference,” in *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*, 2020.
- [16] Liao, Zhipeng et al., “A Memory-Efficient Retrieval Architecture for RAG-Enabled Wearable Medical LLMs-Agents,” in *arXiv preprint arXiv:2510.27107*, 2025.
- [17] A. Montgomerie-Corcoran, P. Toupas, Z. Yu, and C.-S. Bouganis, “SATAY: A streaming architecture toolflow for accelerating YOLO models on FPGA devices,” in *Proceedings of the International Conference on Field Programmable Technology (ICFPT)*, Dec. 2023, pp. 179–187.
- [18] Z. Valadanjoz, H. Daryanavard, and A. Harifi, “High-speed YOLOv4 tiny hardware accelerator for self-driving automotive,” *The Journal of Supercomputing*, vol. 80, no. 5, pp. 6699–6724, Mar. 2024.
- [19] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” in *Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6517–6525, 2017.
- [20] T. Li, Y. Ma, and T. Endoh, “A Systematic Study of Tiny YOLO3 Inference: Toward Compact Brainware Processor With Less Memory and Logic Gate,” *IEEE Access*, vol. 8, pp. 142931–142955, 2020.
- [21] A. Tan, Y. Yang, S. Duan, and L. Wang, “FPGA-Based Convolution Accelerator and Memristor IP Core for Cooperative Acceleration of the YOLO Network,” in *Proceedings of the 2024 4th International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI)*, Nov. 2024, pp. 54–57.
- [22] T. Adiono, A. Putra, N. Sutisna, I. Syafalni, and R. Mulyawan, “Low Latency YOLOv3-Tiny Accelerator for Low-Cost FPGA Using General Matrix Multiplication Principle,” *IEEE Access*, vol. 9, pp. 141890–141913, 2021.